

A. PRETEST: SISTEM MANAJEMEN KARYAWAN

Buatlah sebuah *abstract class* bernama Karyawan yang memiliki atribut nama (bertipe *String* dengan *modifier protected*) beserta sebuah *constructor* untuk menginisialisasinya. Di dalam kelas tersebut, deklarasikan juga sebuah *abstract method* bernama hitungGaji() yang mengembalikan nilai *double* (tanpa *body*). Selanjutnya, buatlah kelas implementasi (*concrete class*) bernama Programmer yang mewarisi kelas Karyawan. Kelas ini memiliki atribut tambahan *private* gajiPokok (bertipe *double*) yang diinisialisasi melalui *constructor*. Implementasikan metode hitungGaji() pada kelas Programmer agar hanya mengembalikan nilai gajiPokok tersebut. Terakhir, uji program Anda dengan membuat kelas Main. Instansiasi satu objek Programmer dan cetak nama beserta total gajinya.

B. LATIHAN PRAKTIKUM: SISTEM PEMBAYARAN DIGITAL (E-PAYMENT)

Instruksi:

Sebagai fondasi awal sistem, buatlah sebuah *abstract class* bernama Pembayaran. Kelas dasar ini harus dapat menyimpan informasi transaksi berupa namaPembayar (*String*) dan nominal (*double*) yang keduanya memiliki hak akses *protected*. Sediakan sebuah *constructor* untuk menginisialisasi kedua atribut tersebut. Selain itu, buatlah *concrete method* tampilkanDetail() untuk mencetak nama pembayar dan nominal transaksi, serta deklarasikan sebuah *abstract method* bernama prosesPembayaran().

Di samping fitur pembayaran, perusahaan mewajibkan adanya standar keamanan untuk seluruh transaksi digital. Oleh karena itu, buatlah sebuah *interface* bernama Keamanan. Di dalam antarmuka ini, deklarasikan sebuah *abstract method* autentikasi() yang mengembalikan nilai *true* (*boolean*).

Setelah fondasi dan standar keamanan siap, tugas Anda selanjutnya adalah membuat dua *concrete class*, yaitu KartuKredit dan EWallet. Kedua kelas ini harus mewarisi kelas Pembayaran sekaligus merealisasikan antarmuka Keamanan. Untuk metode KartuKredit, tambahkan atribut *private* nomorKartu beserta *constructor* yang memanggil *constructor* induk. Saat metode prosesPembayaran() dijalankan untuk kartu kredit, sistem harus menghitung dan mencetak biaya admin sebesar 2% dari nominal, lalu menampilkan total tagihan keseluruhan. Pada proses autentikasi(), cetaklah pesan bahwa autentikasi PIN berhasil dan kembalikan nilai *true*.

Di sisi lain, untuk metode EWallet, tambahkan atribut *private* nomorHP dan buat *constructor* yang sesuai. Berbeda dengan kartu kredit, prosesPembayaran() pada *e-wallet* tidak dikenakan biaya admin sama sekali, sehingga total tagihan sama dengan nominal awal. Untuk proses autentikasi() pada *e-wallet*, sistem cukup mencetak pesan bahwa autentikasi berhasil dan mengembalikan nilai *true*.

Sebagai langkah pengujian terakhir dari arsitektur yang Anda rancang, buatlah kelas Main. Di

dalam program utama ini, buatlah masing-masing satu objek untuk transaksi kartu kredit dan *e-wallet* dengan data acak. Simpan kedua objek tersebut ke dalam sebuah *array* / *arraylist* dengan tipe Pembayaran. Telusurilah *array* / *arraylist* tersebut menggunakan perulangan. Di dalam perulangan, tampilkan detail setiap transaksi. Lakukan pengecekan tipe menggunakan *instanceof* untuk memastikan objek mengimplementasikan Keamanan lalu baru panggil *autentikasi()*. Jika proses autentikasi merespons dengan hasil *true*, maka panggil metode *prosesPembayaran()*.

C. POSTTEST

STUDI KASUS: SISTEM LOGISTIK DAN PENGIRIMAN KARGO TERPADU (LOGISTIK-PRO)

Sebuah perusahaan logistik skala nasional, "Logistik-Pro", membutuhkan sistem perangkat lunak canggih untuk menghitung biaya pengiriman, menerapkan sistem asuransi barang, dan melacak status paket secara *real-time*. Karena setiap armada jalur pengiriman (Darat dan Udara) memiliki regulasi, cara perhitungan tarif, dan fasilitas tambahan yang berbeda, Anda ditugaskan sebagai *Lead Programmer* untuk membangun arsitektur inti dari sistem ini menggunakan konsep *Abstract Class* dan *Interface*.

Sebagai fondasi utama, Anda harus membuat sebuah *abstract class* bernama *LayananPengiriman*. Kelas dasar ini bertugas menyimpan data operasional yang terdiri dari *noResi* (String), *beratBarang* dalam satuan kg (double), dan *jarakTempuh* dalam satuan km (double). Ketiga atribut ini harus terlindungi dengan hak akses *protected* dan wajib diinisialisasi melalui sebuah *constructor*. Di dalam kelas ini, sediakan sebuah method konkret bernama *cetakResi()* yang bertugas menampilkan informasi Nomor Resi, Berat Barang, dan Jarak Tempuh ke layar. Selain itu, Anda harus mendeklarasikan sebuah *abstract method* *hitungOngkosKirim()* yang mengembalikan nilai *double* tanpa parameter apa pun, yang nantinya akan menentukan algoritma tarif masing-masing armada.

Selain kelas dasar, "Logistik-Pro" mewajibkan adanya dua modul tambahan yang berdiri sendiri berupa *interface*. Pertama, buatlah antarmuka bernama *LacakKargo* yang mewajibkan implementasi metode *abstract* *updateStatus(String status)* untuk memperbarui posisi barang, serta metode *cekLokasiTerakhir()* yang mengembalikan nilai *String* berupa posisi terkini. Kedua, buatlah antarmuka bernama *Asuransi* khusus untuk barang-barang berharga. Antarmuka ini harus mewajibkan implementasi metode *hitungPremi(double nilaiBarang)* yang mengembalikan nilai *double*. Untuk memastikan keseragaman dokumen, tambahkan sebuah *default method* (fitur Java 8) bernama *cetakPolis()* di dalam antarmuka *Asuransi* yang langsung mencetak teks: "*Polis Asuransi aktif: Menanggung kehilangan dan kerusakan fisik sebesar 100% dari nilai barang.*"

Setelah fondasi dan modul siap, tugas Anda adalah memetakan armada ke dalam dua *concrete class*. Kelas pertama adalah PengirimanDarat yang mewarisi LayananPengiriman dan **hanya** mengimplementasikan antarmuka LacakKargo (karena jalur darat tidak difasilitasi asuransi sistem). Kelas ini memiliki atribut tambahan *private* berupa jenisTruk (String) dan statusSaatIni (String). Buatlah *constructor* untuk melengkapi semua data induk dan data khusus, serta secara otomatis mengatur statusSaatIni dengan teks awal "*Menunggu Kurir*". Algoritma untuk hitungOngkosKirim() pada jalur darat adalah: $(\text{beratBarang} * 5000) + (\text{jarakTempuh} * 2000)$. Khusus jika jenisTruk yang digunakan adalah "*Tronton*" (abaikan besar/kecil huruf), maka akan dikenakan biaya tambahan tetap sebesar Rp 150.000. Jangan lupa untuk meng-*override* metode dari antarmuka LacakKargo agar status lokasi dapat diperbarui dan dikembalikan nilainya dengan benar.

Kelas armada kedua adalah PengirimanUdara yang mewarisi LayananPengiriman dan mengimplementasikan **kedua** antarmuka sekaligus (LacakKargo dan Asuransi). Kelas ini memiliki atribut tambahan *private* nomorPenerbangan (String), statusSaatIni (String), dan nilaiBarang (double). Buatlah *constructor* yang sesuai dan atur statusSaatIni awalnya menjadi "*Menunggu Jadwal Penerbangan*". Tarif dasar untuk armada udara lebih mahal, dengan algoritma hitungOngkosKirim(): $(\text{beratBarang} * 25000) + (\text{jarakTempuh} * 5000)$. Untuk implementasi antarmuka Asuransi, metode hitungPremi() akan mengembalikan nilai premi sebesar **3% (0.03)** dari total nilaiBarang. Implementasi untuk antarmuka lacak kargo aturannya sama persis seperti kelas sebelumnya.

Sebagai ujian puncak untuk arsitektur ini, buatlah kelas Main untuk mensimulasikan operasional harian. Buatlah satu objek PengirimanDarat (Resi: "DRT-001", berat 50kg, jarak 100km, Truk "Tronton") dan satu objek PengirimanUdara (Resi: "UDR-999", berat 10kg, jarak 800km, Penerbangan "GA-123", nilai barang Rp 5.000.000). Ubah status pengiriman darat menjadi "*Sedang di jalan tol Cipali*", dan pengiriman udara menjadi "*Transit di Bandara Soekarno-Hatta*". Simpan kedua objek tersebut ke dalam sebuah *array polymorphism* bertipe LayananPengiriman[]. Lakukan perulangan pada *array* tersebut untuk: memanggil cetakResi(), mengecek lokasi terakhir dengan melakukan *casting* secara polimorfik, serta menghitung ongkos kirim dasar. Yang paling penting, gunakan operator instanceof untuk mendeteksi apakah suatu armada memiliki fasilitas asuransi. Jika iya, panggil metode cetakPolis(), hitung preminya, lalu tambahkan ke ongkos kirim dasar untuk mendapatkan total akhir. Cetak Total Tagihan untuk setiap pengiriman, dan gunakan garis pemisah agar *output* terminal terlihat rapi dan profesional!